Fixing `std::bit_cast` of types with padding bits

Document number:
P3969R0

Date:
2026-02-20

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:
Jan Schultke <janschultke@gmail.com>

GitHub Issue:
wg21.link/P3969/github

Source:
github.com/eisenwave/cpp-proposals/blob/master/src/bit-cast-padding.cow

---

ABSTRACT: When bit-casting a type containing padding bits to a type with no padding bits, `std::bit_cast` degenerates into an alternative spelling for `std::unreachable` (some exceptions apply). Two viable solutions to the problem are presented: diagnosing `std::bit_cast` and adding a `std::bit_cast_zero_padding` function with alternative behavior, or simply changing the current behavior of `std::bit_cast`.

§ Contents

§ 1. Introduction

BUG: The following use of `std::bit_cast` has undefined behavior at compile time:

```
constexpr auto x = std::bit_cast<__int128>(0.0L); // GCC accepts (x = 0)
```

That is because an 80-bit x87 `long double` has 6 bytes of padding, and it is undefined behavior to map those padding bits onto non-padding bits in the destination type via `std::bit_cast`. [bit.cast] does not disqualify this use of `std::bit_cast` from being a constant expression.

Surprisingly, the undefined behavior in such cases does not depend on the argument. A specialization `std::bit_cast<To, From>` is an alternative spelling for `std::unreachable` if `From` has padding bits and `To` does not, a *degenerate form*. Despite not depending on the argument, the degenerate form of `std::bit_cast` does not violate the *Constraints* or *Mandates* element, leaving the bug undetected. Compilers also have no warning for the degenerate form at the time of writing.

NOTE: If those padding paddings in `From` are all mapped onto `std::byte` or `unsigned char` objects within `To`, the behavior is well-defined.

This behavior is a footgun, and is not very useful. If users wanted a function that always has UB, they should be writing `std::unreachable`, not `std::bit_cast`.

Furthermore, it would be useful if bit-casting between `long double` and a 128-bit integer type was easily possible. After all, reinterpreting floating-point types and integer types is part and parcel of implementing mathematical functions like those in `<cmath>`. It would also be useful if this could be done portably in constant expressions. Another case where the degenerate form

may arise frequently is bit-casting `_BitInt` (supported by Clang as an extension and proposed in [P3666R2]), considering that most `_BitInt` types (at least 7/8) have padding bits.

NOTE: It is possible to implement a proper conversion from `long double` to `__int128`, although it requires multiple steps:

```
// OK because indeterminate bits go into unsigned char:
auto bytes = std::array<unsigned char, 16>(0.0L);
for (int i : { 10, 11, 12, 13, 14, 15 }) bytes[i] = 0;
auto result = std::bit_cast<__int128>(bytes);
```

Another possible workaround is to use a `struct` containing a `__int128 x:80` bit-field, under the assumption those 80 bits line up with those in `long double`.

NOTE: The GCC behavior in the example above is more accurately explained by `long double` having no padding bytes. From a C++ standard perspective, GCC's `long double` is a type with no padding bytes, but six upper bytes that are always zero (assuming any of this behavior is intentional and not just a compiler bug):

```
// OK: uppermost byte is not a padding byte, but is zero
static_assert(bit_cast<array<unsigned char, 16>>(0.0L)[15] == 0);

// UB: forms a long double whose value representation is not valid for t
constexpr auto f = bit_cast<long double>(__int128(-1));
// Passes on GCC, despite bit_cast having set all bytes of f to -1,
// and despite long double having no padding bytes judging by the previo
static_assert(bit_cast<array<unsigned char, 16>>(f)[15] == 0);
```

GCC compiles this code; Clang already rejects both assertions.

2. Design

To address these issues with `std::bit_cast`, there are two viable approaches:

1. Make the degenerate form of `std::bit_cast` ill-formed. Also add a new `std::bit_cast_zero_padding` function which treats padding bits in the source as zero instead of as indeterminate. Other than that, this new function has the same behavior as `std::bit_cast`.
2. Make `std::bit_cast` behave like `std::bit_cast_zero_padding` without adding any new function. This should be done as a DR against C++20.

These are referred to as the *two-function solution* and *single-function solution* below, respectively.

2.1. Advantages of the two-function solution

The single-function solution is problematic because `std::bit_cast` can be used to convert padded types to a byte array without undefined behavior and with zero overhead. Wiping padding bits would add more cost to existing code. With only a single function, there is also no way to opt out of that cost other than using `std::memcpy` instead, and that only works outside of constant evaluation.

Furthermore, if users assumed `std::bit_cast` to clear padding, they may inadvertently access uninitialized memory on older compiler versions, where that behavior is not implemented yet. Perfectly well-defined C++29 code with no erroneous behavior that uses `std::bit_cast` could be copied and pasted into older code bases, and suddenly obtain undefined behavior.

Last but not least, users may be surprised by `std::bit_cast` changing the value of any bits. Conceptually, it is a reinterpretation of existing bits as a new type, and it is desirable to express behavior like zeroing of padding explicitly. This surprising behavior may also sweep developer mistakes under the rug; bit-casting a padded type to an unpadded type may happen unintentionally, and if it was diagnosed, it would inform the user about incorrect assumptions. That often seems more desirable than just zeroing the padding bits and thus silencing any problems.

2.2. Advantages of the single-function solution

The obvious benefit of changing the behavior of `std::bit_cast` is that existing UB in users' code disappears, without any refactoring effort. This would especially be the case if the proposal is treated as a DR against C++20.

Additionally, some may argue that `std::bit_cast_zero_padding` should be the default anyway, considering that it's "safer" to use.

The single-function solution is also easier to implement; it only requires a single `__builtin_bit_cast` intrinsic to be maintained.

2.3. Can't you clear padding bits before bit-casting?

In the discussion of this proposal prior to publication, it was suggested to clear the padding *before* bit-casting. That is, standardizing `__builtin_clear_padding` and using an idiom such as:

```
long double x = /* ... */;
std::clear_padding(x);
std::bit_cast<__int128>(x);
```

However, there are severe problems with this aproach, explained below.

2.3.1. Padding bits are finicky

There are only a few places in the standard where padding bits receive a useful value. For example, zero-initialization is also stated to result in padding bits being zeroed

([dcl.init.general] definition of "zero-initialization"). In most scenarios (e.g. local variables), the padding bits have erroneous or indeterminate value. Even when the padding bits have defined value, lvalue-to-rvalue conversion does not propagate padding bits, and the assignment operator may render them indeterminate or erroneous.

This makes it highly questionable to access padding bits and rely on them having any specific value. If the user forgets to write `std::clear_padding` or falsely assumes that padding bits are already cleared, they could easily acccess uninitialized memory (which may be a security vulnerability).

2.3.2. No padding bits during constant evaluation

Besides the safety issues, the approach of clearing padding bits in the object does not make any sense for constant evaluation. For instance, Clang does not store an object representation for values during constant evaluation. When bit-casting, one is generated “on the fly”.

This would likely mean that `constexpr std::clear_padding` is effectively unimplementable in current compilers.

2.3.3. `std::clear_padding` is not ergonomic for bit-casting

We typically pass large types by reference, even if they are trivially copyable. Assuming we want to cast a type `BigT` to another type `BigU` while clearing padding, the procedure has a lot of steps:

```
__int128 cast(long double x) {
  // 1. Clear padding.
  std::clear_padding(x);
  // 2. Create a variable for holding the result.
  __int128 result;
  // 3. Use std::memcpy to convert the bits.
  //    This is necessary because std::bit_cast ignores the values of
  //    padding bits in the original, so even though we've cleared them,
  //    they would not be propagated.
  std::memcpy(&result, &x, sizeof(__int128));
  // 4. Return the result.
  return result;
}
```

This procedure gets even more complicated when we receive a `const&` or operate on a `std::span<const T>`, in which case we need to create a temporary variable that we can mutate with `std::clear_padding`.

Regardless, this procedure is fairly complex compared to using a `std::bit_cast_zero_padding` function that does it all in one go. All of that complexity yields no advantage; even if `std::clear_padding` was `constexpr`, `std::memcpy` isn't, so `cast` cannot be made `constexpr`.

2.3.4. `std::clear_padding` is less capable

Last but not least, `std::clear_padding` is strictly less capable than `std::bit_cast_zero_padding` because `std::clear_padding` (at least with current compiler technology) is not a viable solution during constant evaluation. However, `std::clear_padding` can be implemented in terms of `std::bit_cast_zero_padding`:

```
template <typename T>
void clear_padding(T& object) {
  // 1. Convert to a byte array.
  //    All the input padding bits are cleared,
  //    and there are not padding bits in a byte array.
  auto zeroed = std::bit_cast_zero_padding<std::array<unsigned char, sizeof(T)>
  // 2. Copy the bytes back into the object.
  //    The bits in the value representation have not been changed,
  //    so this does not change the value of T, only the values of padding bits
  std::memcpy(&object, &zeroed, sizeof(T));
}
```

2.4. Can't you make `std::bit_cast` produce unspecified or erroneous values?

A possible approach would be to make `std::bit_cast` produce unspecified bit values instead of indeterminate bit values. That is, `std::bit_cast<__int128>(0.0L)` would create a `__int128` with 10 predictable bytes and 6 bytes with unspecified value. There are two problems with this idea:

- Since the byte values are now unspecified, UBSan (undefined behavior sanitizer) can no longer diagnose accessing/branching based on the upper 6 bytes as a bug. The bug (possibly CWE-908: Use of Uninitialized Resource) didn't go away, it just became non-conforming to diagnose it with termination.
- This approach should not work for constant evaluation because it would add non-determinism at compile time.

Overall, this design “sweeps the problem under the rug” with little to no benefit to the user.

It is also possible to make the result have erroneous value. However, once again, this approach could not be used to portably bit-cast `long double` to `__int128`, especially not during constant evaluation; the degenerate form of `std::bit_cast` would then always produce erroneous values, so it makes no sense to let it compile in the first place. This solution would only benefit the case of bit-casting to a byte array; perhaps that is worth pursuing, but the only way not to add cost to `std::bit_cast` (with no opt-out) would be to give the bytes an unspecified value that is considered an erroneous value. This provides minimal (if any) benefit, and could be explored in a separate paper; it is a separate issue from the one presented in this paper.

2.5. *Constraints* vs *Mandates* for the two-function solution

The degenerate form of `std::bit_cast` should be diagnosed using a *Mandates* element (that is, `static_assert`). That is because the condition for the degenerate form is relatively complicated and may change in the future. Also, *Constraints* tempts the user to test whether

`bit_cast` is “safe” using `requires`, but this test can have false positives. The detection of the degenerate form would only tell the user whether *all* possible arguments result in undefined behavior.

Conceptually, *Constraints* for `std::bit_cast` should tell the user whether bit-casting is technically feasible due to sizes matching and types being trivially copyable, whereas *Mandates* should catch misuses such as passing consteval-only types or types that result in the degenerate form.

2.6. Requiring `std::bit_cast` UB to be diagnosed in constant expressions

[bit.cast] paragraph 4, bullet 2 explicitly makes indeterminate result bits undefined behavior *inside* `std::bit_cast`, which arguably makes it “library UB”, which is generally not required to be diagnosed during constant evaluation.

I argue that it should be diagnosed, both for the two-function and single-function solution. While `std::bit_cast` is *technically* a library feature, it is spiritually a core language feature, and just acts as a portable spelling for the underlying `__builtin_bit_cast` intrinsic in compilers. Core language UB is generally diagnosed as per [expr.const].

It should be noted that [P0476R1] never motivated this lack of diagnostics, and it is likely wording defect. After all, in the cases where `std::bit_cast` has library UB, it also produces an indeterminate result, and constant expressions do not allow for indeterminate scalar prvalues ([expr.const] definition of "expression,constant"). However, crucially, the library UB taking place inside `std::bit_cast` precedes the [expr.const] policy on what how indeterminate results are treated once a function has returned. Once any library UB happens, everything is UB, so the policy in [expr.const] arguably does not apply.

2.7. Bumping the feature-test macro

For both the two-function and single-function solution, the `__cpp_lib_bit_cast` macro should be bumped:

- For the two-function solution, this lets the user detect the presence of `std::bit_cast_zero_padding`.
- For the single-function solution, this lets the user detect whether they can evaluate `std::bit_cast<__int128>(0.0L)` without undefined behavior.

3. Implementation experience

First, it should be noted that compilers behave radically differently.

EXAMPLE:

```cpp
#include <bit>
struct alignas(4) empty { };
constexpr auto x = std::bit_cast<int>(empty{});
static_assert(x == 0);
```

At the time of writing, GCC and Clang reject the example because it is considered access of an uninitialized byte. MSVC accepts the example, and the assertion passes.

It appears that the proposed behavior of `std::bit_cast_zero_padding` (which is the behavior of `std::bit_cast` in the single-function solution) is already implemented by MSVC, and to a limited extent, by GCC.

There is no implementation experience for the detection of the degenerate form in the two-function solution, and such detection would require compiler support because there exists no way to query which bits or bytes of a type are padding bits, or whether a type has padding bits in the first place.

NOTE: `std::has_unique_object_representations_v<float>` is `true` despite `float` not having padding bits. These false positives make it not suitable for detecting the presence of padding bits.

4. Wording

The changes are relative to [N5032].

DECISION: Based on LEWG feedback, one of the following two sections should be chosen.

4.1. Two-function solution

[version.syn]

Bump the feature-test macro in [version.syn] as follows:

```cpp
#define __cpp_lib_bit_cast ~~201806L~~ 20XXXXL // freestanding, also in <bit>
```

[bit.syn]

Change [bit.syn] as follows:

```cpp
// all freestanding
namespace std {
  // [bit.cast], ~~bit_cast~~ bit-casting
  template<class To, class From>
    constexpr To bit_cast(const From& from) noexcept;
```

```
  template<class To, class From>
    constexpr To bit_cast_zero_padding(const From& from) noexcept;

  […]
}
```

§ **[bit.cast]**

Change [bit.cast] as follows:

~~Function template `bit_cast`~~ Bit-casting **[bit.cast]**

```
template<class To, class From>
  constexpr To bit_cast(const From& from) noexcept;
```

Constraints:

- `sizeof(To) == sizeof(From)` is `true`;
- `is_trivially_copyable_v<To>` is `true`;
- `is_trivially_copyable_v<From>` is `true`.

Mandates:

- Neither `To` nor `From` are consteval-only types ([basic.types.general])~~.~~ ; and
- for some argument of type `From`, the result of the function call expression is well-defined.

Constant When: `To`, `From`, and the types of all subobjects of `To` and `From` are types `T` such that:

- `is_union_v<T>` is `false`;
- `is_pointer_v<T>` is `false`;
- `is_member_pointer_v<T>` is `false`;
- `is_volatile_v<T>` is `false`;
- `T` has no non-static data members of reference type.

Returns: An object of type `To`. Implicitly creates objects nested within the result ([intro.object]). Each bit of the value representation of the result is equal to the corresponding bit in the object representation of `from`. Padding bits of the result are unspecified. For the result and each object created within it, if there is no value of the object's type corresponding to the value representation produced, the behavior is undefined. If there are multiple such values, which value is produced is unspecified. A bit in the value representation of the result is indeterminate if it does not correspond to a bit in the value representation of `from` or corresponds to a bit for which the smallest enclosing object is not within its lifetime or has an indeterminate value ([basic.indet]). A bit in the value representation of the result is erroneous if it corresponds to a bit for which the smallest enclosing object has

an erroneous value. For each bit b in the value representation of the result that is indeterminate or erroneous, let u be the smallest object containing that bit enclosing b:

- If u is of unsigned ordinary character type or `std::byte` type, u has an indeterminate value if any of the bits in its value representation are indeterminate, or otherwise has an erroneous value.
- Otherwise, if b is indeterminate, the behavior is undefined.
- Otherwise, the behavior is erroneous, and the result is as specified above.

The result does not otherwise contain any indeterminate or erroneous values.

Remarks: A function call expression whose behavior is undefined as per the *Returns* element is not a core constant expression ([expr.const]).

Append the following declaration to [bit.cast]:

```
template<class To, class From>
  constexpr To bit_cast_zero_padding(const From& from) noexcept;
```

Effects: Equivalent to `bit_cast<To>(from)`, except that if a bit b in the value representation of the result does not correspond to a bit in the value representation of `from`, b is zero, not indeterminate.

[*Example*: The following example assumes that `sizeof(S) == 1` is `true`.

```
struct S { };
void f() {
  bit_cast<char8_t>(S{});                // error: bit_cast<char8_t, S> is
  bit_cast<unsigned char>(S{});          // OK, returns indeterminate value
  bit_cast_zero_padding<char8_t>(S{});   // OK, returns char8_t{0}
}
```

— *end example*]

4.2. Single-function solution

[version.syn]

Bump the feature-test macro in [version.syn] as follows:

```
#define __cpp_lib_bit_cast ~~201806L~~ 20XXXXL // freestanding, also in <bit>
```

[bit.cast]

Change [bit.cast] as follows:

Function template `bit_cast` [bit.cast]

```
template<class To, class From>
  constexpr To bit_cast(const From& from) noexcept;
```

Constraints:

- `sizeof(To) == sizeof(From)` is `true`;
- `is_trivially_copyable_v<To>` is `true`;
- `is_trivially_copyable_v<From>` is `true`.

Mandates: Neither `To` nor `From` are consteval-only types ([basic.types.general]).

Constant When: `To`, `From`, and the types of all subobjects of `To` and `From` are types `T` such that:

- `is_union_v<T>` is `false`;
- `is_pointer_v<T>` is `false`;
- `is_member_pointer_v<T>` is `false`;
- `is_volatile_v<T>` is `false`;
- `T` has no non-static data members of reference type.

Returns: An object of type `To`. Implicitly creates objects nested within the result ([intro.object]). Each bit of the value representation of the result is equal to the corresponding bit in the object representation of `from`. Padding bits of the result are unspecified. For the result and each object created within it, if there is no value of the object's type corresponding to the value representation produced, the behavior is undefined. If there are multiple such values, which value is produced is unspecified. A bit in the value representation of the result is ~~indeterminate~~ zero if it does not correspond to a bit in the value representation of `from` ~~or~~, and is indeterminate if it corresponds to a bit for which the smallest enclosing object is not within its lifetime or has an indeterminate value ([basic.indet]). A bit in the value representation of the result is erroneous if it corresponds to a bit for which the smallest enclosing object has an erroneous value. For each bit b in the value representation of the result that is indeterminate or erroneous, let u be the smallest object containing that bit enclosing b:

- If u is of unsigned ordinary character type or `std::byte` type, u has an indeterminate value if any of the bits in its value representation are indeterminate, or otherwise has an erroneous value.
- Otherwise, if b is indeterminate, the behavior is undefined.
- Otherwise, the behavior is erroneous, and the result is as specified above.

The result does not otherwise contain any indeterminate or erroneous values.

> *Remarks*: A function call expression whose behavior is undefined as per the *Returns* element is not a core constant expression ([expr.const]).

§ 5. References

[N5032] *Thomas Köppe.* Working Draft, Programming Languages — C++ (2025-12-15)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf

[P0476R1] *JF Bastien.* Bit-casting object representations (2016-11-11)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0476r1.html

[P3666R2] *Jan Schultke.* Bit-precise integers (2025-12-14)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3666r2.pdf